Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №3 по курсу

«Операционные системы»

Группа: М8О-208БВ-25

Студент: Лунева Л.В.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 09.11.26

Москва, 2026

Постановка задачи

Вариант 13.

Цель работы: приобрести практические навыки организации межпроцессного взаимодействия с использованием именованной общей памяти, отображения памяти и семафоров в ОС Linux.

Задание: переработать программу первой лабораторной работы, заменив обмен через неименованные каналы обменом через общую память. Родительский процесс должен создавать два дочерних процесса, представленных отдельными исполняемыми программами, принимать строки из стандартного ввода и выводить результат обработки в стандартный вывод.

В варианте 13 процесс child1 переводит латинские буквы в нижний регистр. Процесс child2 заменяет пробельные символы на «_», сохраняя переводы строк и пустые строки. Обработка продолжается до конца входного потока (EOF).

Объект общей памяти необходимо создавать функцией shm_open и отображать в адресное пространство процессов с помощью mmap. Для согласования чтения и записи используются именованные семафоры. Имена объектов должны различаться у одновременно работающих экземпляров программы. Необходимо обрабатывать ошибки создания ресурсов, запуска процессов и обмена данными.

Программа написана на языке Си. Ввод и вывод выполняются через read и write. Функция snprintf используется только для формирования имён в массивах памяти.

Общий метод и алгоритм решения

Программа состоит из файлов common.h, parent.c, child1.c и child2.c. Заголовочный файл содержит описание общего буфера и вспомогательные функции. Makefile собирает три исполняемых файла: parent, child1 и child2.

Shared memory — это объект, содержащий общие для процессов данные. Memory mapping — способ отобразить этот объект в виртуальное адресное пространство процесса. Каждый процесс получает собственный указатель на отображение, но при MAP_SHARED изменения относятся к одному и тому же объекту. Передача данных между процессами выполняется через обращения к полям структуры, без read и write для самого общего буфера.

Структура shared содержит поле length типа uint32_t и массив data. При SHM_SIZE = 4096 поле длины занимает 4 байта, а массив данных — 4092 байта. Значение UINT32_MAX используется как STOP_MARKER и сообщает об окончании ввода. Размер 4096 выбран для буфера программы и не ограничивает общую длину входного текста.

Порядок работы процессов

Родитель формирует имена /lab3-shm-<PID>, /lab3-sem1-<PID>, /lab3-sem2-<PID> и /lab3-sem3-<PID>. PID различает одновременно работающие экземпляры. Флаг O_EXCL не позволяет незаметно открыть уже существующий объект при его создании.

Затем родитель создаёт объект общей памяти, задаёт его размер через ftruncate и выполняет mmap с PROT_READ | PROT_WRITE и MAP_SHARED. После успешного отображения файловый дескриптор закрывается: отображение остаётся доступным до munmap.

Создаются три семафора с начальным значением 0. Родитель запускает child1 и child2 с помощью fork и execv, передавая им имена общей памяти и нужных семафоров через аргументы командной строки. После execv дети самостоятельно открывают объекты по этим именам и создают свои отображения.

Назначение семафоров

По sem1 родитель сообщает child1 о новой порции данных. По sem2 child1 сообщает child2 об окончании перевода в нижний регистр. По sem3 child2 сообщает родителю о готовности результата.

sem_wait уменьшает положительное значение семафора на единицу, а при нулевом значении ожидает. sem_post увеличивает значение и позволяет ожидающему процессу продолжить работу. Вспомогательная функция sem_wait_retry повторяет ожидание, если оно прервано сигналом с ошибкой EINTR.

Обработка очередной порции

1. Родитель читает до 4092 байт из STDIN непосредственно в sh->data и записывает полученную длину в sh->length. После sem_post(sem1) он ожидает sem3 и не изменяет буфер.

2. Процесс child1 проходит ожидание sem1, проверяет длину и применяет tolower к каждому байту данных. После обработки он вызывает sem_post для sem2.

3. Процесс child2 проходит ожидание sem2 и заменяет пробельные символы, кроме \n, на «_». Затем он вызывает sem_post для sem3.

4. Родитель дожидается sem3 и выводит порцию целиком. При частичной записи он дописывает оставшиеся байты. После вывода он читает следующую порцию. Строки произвольной длины обрабатываются за несколько циклов.

В каждый момент с общим буфером работает один участник. Семафоры определяют порядок доступа, а данные остаются в общей памяти. Преобразование регистра выполняется побайтово для латинских букв.

Завершение и используемые функции

При EOF родитель передаёт STOP_MARKER через sem1. Child1 пересылает его через sem2 и завершается. Child2 подтверждает получение через sem3 и тоже завершается. В терминале EOF можно передать сочетанием Ctrl+D на пустой строке.

Родитель последовательно ждёт оба процесса через waitpid и проверяет их статусы. Затем sem_trywait проверяет подтверждение EOF. При освобождении ресурсов родитель удаляет только те имена IPC, которые создал сам.

При обработке данных sem_timedwait ждёт sem3 до одной секунды, затем waitpid с WNOHANG проверяет, живы ли дочерние процессы. При ошибке оставшиеся процессы останавливаются, а ресурсы освобождаются. Родитель игнорирует SIGPIPE, поэтому закрытие канала вывода обрабатывается как ошибка EPIPE при вызове write.

Основные функции POSIX

shm_open(name, flags, mode) — создаёт или открывает именованный объект общей памяти и возвращает файловый дескриптор.

ftruncate(fd, length) — устанавливает размер объекта, чтобы в нём поместилась структура shared.

mmap(addr, length, prot, flags, fd, offset) — отображает объект по файловому дескриптору в адресное пространство процесса; при ошибке возвращает MAP_FAILED.

munmap(addr, length) — удаляет отображение из адресного пространства текущего процесса.

close(fd) и shm_unlink(name) — закрывают файловый дескриптор и удаляют имя объекта общей памяти соответственно.

sem_open(name, flags, ...) — создаёт или открывает именованный семафор; при ошибке возвращает SEM_FAILED.

sem_wait, sem_timedwait, sem_trywait — ожидают семафор без ограничения времени, до заданного срока или проверяют его без ожидания соответственно.

sem_post(sem) — увеличивает значение семафора и уведомляет следующего участника.

sem_close(sem), sem_unlink(name) — закрывают семафор для текущего процесса и удаляют его имя.

getpid(), fork(), execv(path, argv) — получают PID, создают дочерний процесс и запускают в нём отдельную программу.

read(fd, buf, count), write(fd, buf, count) — читают входные данные и выводят результат либо сообщение об ошибке.

waitpid(pid, status, options), kill(pid, signal) — ожидают или проверяют завершение ребёнка и отправляют сигнал процессу.

clock_gettime, sigaction и _exit — получают время для срока ожидания, задают реакцию на сигнал и завершают процесс.

Код программы

В листингах приведены исходные файлы и правила сборки трёх исполняемых программ.

common.h

```
#ifndef COMMON_H
#define COMMON_H

#include <stdint.h>
#include <stdlib.h>
#include <unistd.h>
#include <errno.h>
#include <fcntl.h>
#include <semaphore.h>
#include <sys/mman.h>

#define SHM_SIZE 4096

#define STOP_MARKER UINT32_MAX

struct shared {
    uint32_t length;
    char data[SHM_SIZE - sizeof(uint32_t)];
};

static inline void print_msg(const char *msg) {
    size_t len = 0;
    while (msg[len] != '\0')
        ++len;
    write(STDERR_FILENO, msg, len);
}

static inline void print_error(const char *msg) {
    print_msg(msg);
    _exit(EXIT_FAILURE);
}

static inline struct shared *map_shared(const char *name) {
    int shm = shm_open(name, O_RDWR, 0);
    if (shm == -1)
        return NULL;

    void *addr = mmap(NULL, sizeof(struct shared), PROT_READ | PROT_WRITE,
                      MAP_SHARED, shm, 0);

    close(shm);

    return addr == MAP_FAILED ? NULL : addr;
}

static inline int sem_wait_retry(sem_t *sem) {
    int result;
    while ((result = sem_wait(sem)) == -1 && errno == EINTR)
        ;
    return result;
}

#endif
```

parent.c

```
#include "common.h"
#include <stdbool.h>
#include <stdio.h>
#include <signal.h>
#include <time.h>
#include <sys/wait.h>

static char shm_name[64];
static char sem1_name[64];
static char sem2_name[64];
static char sem3_name[64];

static struct shared *sh = NULL;
static bool shm_created = false;
static sem_t *sem1 = SEM_FAILED;
static sem_t *sem2 = SEM_FAILED;
static sem_t *sem3 = SEM_FAILED;

static pid_t child1 = -1;
static pid_t child2 = -1;

static void release_resources(void) {
    if (sem1 != SEM_FAILED) {
        sem_close(sem1);
        sem_unlink(sem1_name);
    }
    if (sem2 != SEM_FAILED) {
        sem_close(sem2);
        sem_unlink(sem2_name);
    }
    if (sem3 != SEM_FAILED) {
        sem_close(sem3);
        sem_unlink(sem3_name);
    }
    if (sh != NULL)
        munmap(sh, sizeof(*sh));
    if (shm_created)
        shm_unlink(shm_name);
}

static pid_t wait_child(pid_t pid, int *status, int options) {
    pid_t result;
    do {
        result = waitpid(pid, status, options);
    } while (result == -1 && errno == EINTR);
    return result;
}

static void fail(const char *msg) {
    print_msg(msg);

    if (child1 > 0) {
        kill(child1, SIGKILL);
        wait_child(child1, NULL, 0);
    }
    if (child2 > 0) {
        kill(child2, SIGKILL);
        wait_child(child2, NULL, 0);
    }

    release_resources();
    _exit(EXIT_FAILURE);
}

static sem_t *create_sem(const char *name) {
    sem_t *sem = sem_open(name, O_CREAT | O_EXCL, 0600, 0);
    if (sem == SEM_FAILED)
        fail("parent: failed to create semaphore\n");
    return sem;
}
```

```
static void check_child(pid_t *pid, int options, const char *error) {
    int status;
    pid_t result = wait_child(*pid, &status, options);
    if (result == -1)
        fail("parent: failed to wait for child\n");
    if (result == 0)
        return;

    *pid = -1;
    if (options == WNOHANG || !WIFEXITED(status) || WEXITSTATUS(status) != 0)
        fail(error);
}

static void wait_for_result(void) {
    while (1) {
        struct timespec deadline;
        if (clock_gettime(CLOCK_REALTIME, &deadline) == -1)
            fail("parent: failed to get time\n");
        deadline.tv_sec += 1;

        if (sem_timedwait(sem3, &deadline) == 0)
            return;

        if (errno == EINTR)
            continue;
        if (errno != ETIMEDOUT)
            fail("parent: failed to wait for semaphore\n");

        check_child(&child1, WNOHANG, "parent: child1 terminated unexpectedly\n");
        check_child(&child2, WNOHANG, "parent: child2 terminated unexpectedly\n");
    }
}

static void write_all(const char *data, size_t length) {
    while (length > 0) {
        ssize_t written = write(STDOUT_FILENO, data, length);
        if (written < 0 && errno == EINTR)
            continue;
        if (written <= 0)
            fail("parent: failed to write to stdout\n");
        data += written;
        length -= (size_t)written;
    }
}

int main(void) {
    struct sigaction action = { .sa_handler = SIG_IGN };
    if (sigemptyset(&action.sa_mask) == -1 ||
        sigaction(SIGPIPE, &action, NULL) == -1)
        fail("parent: failed to ignore SIGPIPE\n");

    int id = (int)getpid();
    snprintf(shm_name, sizeof(shm_name), "/lab3-shm-%d", id);
    snprintf(sem1_name, sizeof(sem1_name), "/lab3-sem1-%d", id);
    snprintf(sem2_name, sizeof(sem2_name), "/lab3-sem2-%d", id);
    snprintf(sem3_name, sizeof(sem3_name), "/lab3-sem3-%d", id);

    int shm = shm_open(shm_name, O_RDWR | O_CREAT | O_EXCL, 0600);
    if (shm == -1)
        fail("parent: failed to create shared memory\n");
    shm_created = true;

    if (ftruncate(shm, sizeof(struct shared)) == -1) {
        close(shm);
        fail("parent: failed to resize shared memory\n");
    }

    void *addr = mmap(NULL, sizeof(struct shared), PROT_READ | PROT_WRITE,
                      MAP_SHARED, shm, 0);
    close(shm);
    if (addr == MAP_FAILED)
        fail("parent: failed to map shared memory\n");
    sh = addr;
```

```
    sem1 = create_sem(sem1_name);
    sem2 = create_sem(sem2_name);
    sem3 = create_sem(sem3_name);

    child1 = fork();
    if (child1 == -1)
        fail("parent: failed to spawn child1\n");

    if (child1 == 0) {
        char *const args[] = {"child1", shm_name, sem1_name, sem2_name, NULL};
        execv("./child1", args);
        print_error("parent: failed to exec child1\n");
    }

    child2 = fork();
    if (child2 == -1)
        fail("parent: failed to spawn child2\n");

    if (child2 == 0) {
        char *const args[] = {"child2", shm_name, sem2_name, sem3_name, NULL};
        execv("./child2", args);
        print_error("parent: failed to exec child2\n");
    }

    ssize_t bytes;
    while ((bytes = read(STDIN_FILENO, sh->data, sizeof(sh->data))) != 0) {
        if (bytes < 0 && errno == EINTR)
            continue;
        if (bytes < 0)
            fail("parent: failed to read from stdin\n");

        sh->length = (uint32_t)bytes;

        if (sem_post(sem1) == -1)
            fail("parent: failed to post semaphore\n");
        wait_for_result();
        write_all(sh->data, (size_t)bytes);
    }

    sh->length = STOP_MARKER;
    if (sem_post(sem1) == -1)
        fail("parent: failed to post semaphore\n");
    check_child(&child1, 0, "parent: child1 terminated unexpectedly\n");
    check_child(&child2, 0, "parent: child2 terminated unexpectedly\n");
    if (sem_trywait(sem3) == -1)
        fail("parent: missing EOF acknowledgement\n");

    release_resources();
    return 0;
}
```

child1.c

```
#include "common.h"
#include <ctype.h>

int main(int argc, char **argv) {
    if (argc != 4)
        print_error("usage: child1 <shm> <sem_in> <sem_out>\n");

    struct shared *sh = map_shared(argv[1]);
    if (sh == NULL)
        print_error("child1: failed to map shared memory\n");

    sem_t *in = sem_open(argv[2], 0);
    if (in == SEM_FAILED)
        print_error("child1: failed to open input semaphore\n");

    sem_t *out = sem_open(argv[3], 0);
    if (out == SEM_FAILED)
        print_error("child1: failed to open output semaphore\n");

    while (1) {

        if (sem_wait_retry(in) == -1)
            print_error("child1: failed to wait for semaphore\n");

        if (sh->length == STOP_MARKER) {

            if (sem_post(out) == -1)
                print_error("child1: failed to post semaphore\n");
            break;
        }

        if (sh->length > sizeof(sh->data))
            print_error("child1: invalid length in shared memory\n");

        for (uint32_t i = 0; i < sh->length; ++i)
            sh->data[i] = tolower((unsigned char)sh->data[i]);

        if (sem_post(out) == -1)
            print_error("child1: failed to post semaphore\n");
    }

    sem_close(in);
    sem_close(out);
    munmap(sh, sizeof(*sh));

    return 0;
}
```

child2.c

```
#include "common.h"
#include <ctype.h>

int main(int argc, char **argv) {
    if (argc != 4)
        print_error("usage: child2 <shm> <sem_in> <sem_out>\n");

    struct shared *sh = map_shared(argv[1]);
    if (sh == NULL)
        print_error("child2: failed to map shared memory\n");

    sem_t *in = sem_open(argv[2], 0);
    if (in == SEM_FAILED)
        print_error("child2: failed to open input semaphore\n");

    sem_t *out = sem_open(argv[3], 0);
    if (out == SEM_FAILED)
        print_error("child2: failed to open output semaphore\n");

    while (1) {

        if (sem_wait_retry(in) == -1)
            print_error("child2: failed to wait for semaphore\n");

        if (sh->length == STOP_MARKER) {
            if (sem_post(out) == -1)
                print_error("child2: failed to post semaphore\n");
            break;
        }

        if (sh->length > sizeof(sh->data))
            print_error("child2: invalid length in shared memory\n");

        for (uint32_t i = 0; i < sh->length; ++i) {
            if (isspace((unsigned char)sh->data[i]) && sh->data[i] != '\n')
                sh->data[i] = '_';
        }

        if (sem_post(out) == -1)
            print_error("child2: failed to post semaphore\n");
    }

    sem_close(in);
    sem_close(out);
    munmap(sh, sizeof(*sh));

    return 0;
}
```

Makefile

```
CC = gcc
CFLAGS = -Wall -Wextra

all: parent child1 child2

parent: parent.c common.h
    $(CC) $(CFLAGS) -o parent parent.c

child1: child1.c common.h
    $(CC) $(CFLAGS) -o child1 child1.c

child2: child2.c common.h
    $(CC) $(CFLAGS) -o child2 child2.c

clean:
    rm -f parent child1 child2
```

Протокол работы программы

Программа проверена в Linux. GCC собрал её без предупреждений, в том числе с флагами -Wall, -Wextra, -Wpedantic и -Werror. Команды printf передают EOF при закрытии входного канала.

Сборка

```
$ make -B
gcc -Wall -Wextra -o parent parent.c
gcc -Wall -Wextra -o child1 child1.c
gcc -Wall -Wextra -o child2 child2.c
```

Тест 1 Перевод регистра и замена пробела

```
$ printf 'HELLO World\n' | ./parent
hello_world
```

Латинские буквы переведены в нижний регистр, а пробел заменён на подчёркивание.

Тест 2 Несколько строк и табуляция

```
$ printf 'First Line\nSecond     LINE\twith TAB\nMixED CaSe 123   !!!\n' | ./parent
first_line
second_____line_with_tab
mixed_case_123___!!!
```

Каждый пробел и символ табуляции заменён отдельно. Переводы строк, цифры и знаки препинания сохранены.

Тест 3 Пустая строка внутри входного потока

```
$ printf 'ONE\n\nTWO\n' | ./parent
one

two
```

Пустая строка сохранена. Обработка продолжается до EOF, поэтому строка TWO также попадает в результат.

Тест 4 Отсутствие входных данных

```
$ ./parent < /dev/null
$ echo $?
0
```

Программа ничего не выводит и штатно завершает дочерние процессы.

Проверка границы буфера и запуска процессов

Тест 5 Строка длиной 4092 символа

Во входном потоке расположены 4092 символа A, затем перевод строки и строка HELLO World. Таким образом, перевод строки следует сразу за полной порцией данных. Python используется только для подготовки входа и сравнения результата теста.

```
$ python3 - <<'PYTEST'
import subprocess
source = b'A' * 4092 + b'\nHELLO World\n'
expected = b'a' * 4092 + b'\nhello_world\n'
p = subprocess.run(['./parent'], input=source,
                   capture_output=True, timeout=5)
print('code:', p.returncode)
print('bytes:', len(p.stdout))
print('match:', p.stdout == expected)
print('stderr:', repr(p.stderr))
PYTEST
code: 0
bytes: 4105
match: True
stderr: b''
```

Все 4105 байт результата получены. Перевод строки на границе порций не приводит к преждевременному завершению программы.

Тест 6 Отсутствует исполняемый файл child1

```
$ mv child1 child1.bak
$ printf 'test\n' | ./parent
parent: failed to exec child1
parent: child1 terminated unexpectedly
$ echo $?
1
$ mv child1.bak child1
```

Родитель обнаружил ошибку execv, остановил второй дочерний процесс, освободил ресурсы и завершился с кодом 1.

Проверка завершения и очистки ресурсов

Тест 7 Задержка child2 при EOF

Временная обёртка откладывает запуск child2 на две секунды. Родитель ждёт его завершения и возвращает код 0.

```
$ (
    mv child2 child2.real
    trap 'mv -f child2.real child2' EXIT
    printf '#!/bin/sh\nsleep 2\nexec ./child2.real "$@"\n' > child2
    chmod +x child2
    ./parent < /dev/null
    echo $?
)
0
```

Тест 8 Закрытый канал вывода

Читающий конец stdout закрывается до подачи данных. Родитель сообщает об ошибке и освобождает все четыре объекта IPC.

```
$ python3 - <<'PYTEST'
from pathlib import Path
from subprocess import PIPE, Popen
p = Popen(['./parent'], stdin=PIPE, stdout=PIPE, stderr=PIPE)
p.stdout.close()
_, err = p.communicate(b'A B\n', timeout=5)
names = ('lab3-shm', 'sem.lab3-sem1',
         'sem.lab3-sem2', 'sem.lab3-sem3')
left = [n for n in names if Path(f'/dev/shm/{n}-{p.pid}').exists()]
print(err.decode().strip())
print(p.returncode, left)
PYTEST
parent: failed to write to stdout
1 []
```

Тест 9 Занятое имя ресурса

Заранее созданные объекты общей памяти и семафора проверены в отдельных запусках. В обоих случаях программа вернула код 1 и сохранила существовавший объект. При конфликте имени семафора программа также удалила ресурсы, созданные до ошибки.

Вывод

В работе реализован обмен данными между тремя процессами через отображаемую общую память POSIX. Три именованных семафора задают порядок доступа к буферу. Проверки подтвердили обработку строк произвольной длины, сохранение пустых строк, штатное завершение при EOF и очистку ресурсов при ошибках запуска и вывода.